SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA0309	**Course Title:**	Data Structures
CO Assessed:	CO4 – Articulation (BL3)	**Assessment:**	Assessment Tool 3 – Comparative Analysis
Weightage:	30% of CIA	**Total Marks:**	100
CO4 Statement:	*Apply hashing strategies for efficient data storage and sorting algorithms for arrangement of data.*		
Student Name:	B.Sathvika	**Reg. No.:**	192525224
Section / Batch:	2nd year AIML	**Date:**	28/08/2026

Code of Conduct

I B.Sathvika (192525224) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: B.Sathvika

Instructions

- This test contains 5 Comparative Analysis questions. Total: 100 Marks.
- When a student answer using comparative analysis should be based on four requirements: time complexity, space complexity advantages & limitations, real-world scenarios and operations.
- No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
Hashing	Linear Probing & Separate Chaining	1	20
Sorting	Merge Sort & Quick Sort	2	20
Hashing	Quadratic Probing and Double Hashing	3	20
Sorting Algorithms	Complexity Analysis	4	20
Quick Sort	Recursive and Iterative implementations	5	20
	GRAND TOTAL:		100 Marks

Annexure A – Comparative Analysis

1. Compare Linear Probing and Separate Chaining for collision handling in hashing based on: (20 Marks)
   a) Clustering effect
   b) Memory usage
   c) Performance under high load factor
   d) Which method is more suitable for large-scale applications and why?
2. Compare quick sort and merge sort for sorting a large linked list of 10 million nodes. Analyze memory requirements (in-place vs. extra space), cache behavior, and time complexity. Which algorithm is preferable for linked lists and why? How does the answer change for arrays? (20 Marks)
3. Compare Quadratic Probing and Double Hashing in terms of: (20 Marks)
   a) Clustering issues
   b) Collision resolution efficiency
   c) Suitability for dense hash tables
4. Compare Best-case and Worst-case time complexity of sorting algorithms based on the following: (20 Marks)
   a) Practical significance
   b) Impact on algorithm selection
5. Compare Recursive and Iterative implementations of Quick Sort in terms of: (20 Marks)
   a) Space complexity (stack usage)
   b) Ease of implementation
   c) Risk of stack overflow

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Concept Understanding	20	Demonstrates complete and accurate understanding of all data structures with advanced insights	Shows clear understanding with minor gaps	Basic understanding; some misconceptions present	Limited or incorrect understanding
Comparative Analysis Depth	20	Thorough, multi-dimensional comparison with strong justification and critical evaluation	Good comparison with relevant factors considered	Limited comparison; lacks depth or justification	Minimal or incorrect comparison
Use of Parameters (Time/Space/Operations)	30	All parameters analysed accurately with complexity discussion	Most parameters covered correctly	Few parameters considered; some inaccuracies	Parameters missing or incorrect
Critical Thinking	20	Strong justification of why one structure is better in given contexts	Some justification present	Limited reasoning	No critical reasoning
Clarity	10	Exceptionally well-structured, logical flow, clear tables/diagrams	Well-organized with minor issues	Some organization issues; lacks clarity	Poor structure and readability

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

CO4 - Assessment - 03

1) Compare linear Probing and Separate chaining for Collision Handling in Hashing

Introduction:

Hashing is a technique used to store and retrieve data efficiently using a hash table. when two keys produce the same hash index, a collision occurs. linear Probing and Separate chaining are two common methods used to resolve collisions.

a) Clustering Effect:

Linear Probing:

linear probing suffers from primary clustering when consecutive positions become occupied, new elements tend to form long groups of occupied slots. This increases the number of comparisions required to find an element.

Separate chaining:

Separate chaining does not suffer from primary clustering becacuse each hash table position maintains a separate linked list or chain. colliding elements are stored in the same chain.

b) Memory Usage:

Linear Probing:

Linear probing stores all elements directly inside the hash table. therefore, it requires less additional memory.

Separate chaining:

Separate chaining requires extra memory for

linked-list nodes or other data structures used to store collided elements. Hence, its memory requirement is higher.

c) Performance under High Load Factor

Linear Probing:
Its performance decreases significantly when the load factor becomes high. More positions become occupied, increasing the probability of long probe sequences.

Separate chaining:
Separate chaining generally handles high load factors better because multiple elements can be stored in the same bucket. However, very long chains can also reduce search performance.

d) Suitable Method for Large-scale Applications:

For large-scale applications, separate chaining is generally more suitable because it can handle collisions and high load factors more flexibly. It does not require all elements to fit into empty table positions and avoids primary clustering.

Conclusion:

Linear probing is simple and memory-efficient, but its performance can decrease considerably at high load factors. Separate chaining uses more memory but provides better flexibility and collision handling, making it suitable for large-scale applications.

Compare Quick Sort and Merge Sort for Sorting a Large Linked List of 10 Million Nodes

Introduction:

Sorting a linked list containing 10 million nodes requires an algorithm that efficiently handles sequential data and memory limitations. Quick Sort and Merge Sort have different characteristics when applied to linked lists

Memory Requirements:

Quick Sort:

Quick Sort can be implemented with relatively little extra memory if partitioning is performed carefully. However, recursive calls require stack space. In the worst case, the recursion depth can become $O(n)$.

Merge Sort:

Merge Sort requires extra space for recursion, but merging linked lists can be performed without creating another complete array. Therefore, it is highly suitable for linked lists.

Cache Behavior:

Quick Sort:

Quick Sort generally has good cache performance for arrays because it accesses nearby memory locations. However, linked-list nodes may be scattered throughout memory, reducing the cache advantage.

Merge Sort:

Merge Sort accesses linked-list nodes sequentially while splitting and merging. This makes it more

suitable for linked lists, although cache performance depends on how the nodes are allocated in memory.

Time complexity:

Algorithm	Best case	Average case	worst case
Quick sort	O(nlogn)	O(nlogn)	$O(n^2)$
Merge sort	O(n logn)	O(n log n)	O(n log n)

Conclusion:

For a linked list containing 10 million nodes, Merge sort is generally the better choice. For arrays, Quick sort is often preferred when average-case performance and low extra memory are important, while Merge sort is preferred when guaranteed O(n log n) performance is required.

3 Compare Quadratic Probing and Double Hashing

Introduction:

Quadratic probing and double hashing are open-addressing techniques used to resolve collisions in hash tables. Both methods search for another available position when the original hash position is occupied.

a) Clustering Issues

Quadratic Probing:

Quadratic probing reduces primary clustering compared with linear probing. However, it can still suffer from secondary clustering, where keys having the same initial hash value follow the same probing sequence.

Double Hashing:

Double hashing greatly reduces clustering because a second hash function determines the step size. Different keys are more likely to follow different probing sequences.

b) Collision Resolution Efficiency:

Quadratic probing:

It provides better collision resolution than linear probing and is relatively simple to implement. However, secondary clusterring can still affect performance.

Double Hashing:

Double hashing generally provides more efficient collisions resolution because it distributes probes more uniformly throughout the table.

c) Suitability for Dense Hash Tables:

Quadratic Probing:

Its performance decreases as the table becomes very full. It is therefore less suitable for extremely dense hash tables.

Double Hashing:

Double hashing performs better than quadratic

probing when the table becomes relatively dense.

Comparision:

Feature	Quadratic Probing	Double Hashing
primary clustering	Reduced	very low
Secondary clustering	present	Greatly reduced
collision resolution	Good	very good
Implementation	simple	More complex
Dense tables	Moderate suitability	Better suitability

Conclusion:

Double hashing is generally more efficient than quadratic probing because it reduces clustering and distributes key more evenly

4. Compare Best-case and worst-case Time Complexity of sorting Algorithms.

Introduction:

Time complexity describes how the running time of an algorithm changes as the input size increases Best-case and worst case complexities help us understand the performance of sorting alogorithms under different input conditions.

a) practical significance:

Best - case complexity represents the minimum time required by an algorithm for a favorable input arrangement. It shows how efficiently an algorithm can perform when the input is already suitable for the algorithm.

worst - case complexity represents the maximum time required for an input arrangement that causes the algorithm to perform poorly. It is especially important when applications need predictable performance.

For example: Quick Sort has:

- Best case: $O(n \log n)$
- Average case: $O(n \log n)$
- worst case: $O(n^2)$

Merge Sort has:

- Best case: $O(n \log n)$
- Average case: $O(n \log n)$
- worst case: $O(n \log n)$

b) Impact on Algorithm selection:

Best-case complexity is useful when input data is usually well organized. However, selecting an algorithm only based on its best case can be risky

worst-case complexity is important when the application must guarantee reasonable performance for all inputs.

General Comparision:

Sorting Algorithm	Best case	worst case
Bubble Sort	$O(n)$	$O(n^2)$
Insertion Sort	$O(n)$	$O(n^2)$

Quick sort	O(nlogn)	$O(n^2)$
Merge sort	O(nlogn)	O(nlogn)
Heap sort	O(nlogn)	O(nlogn)

5 Compare Recursive and Iterative Implementations of Quick sort.

Introduction:

Quick sort can be implemented using either recursion or iteration. Both approaches use the partitioning principle of Quick sort, but they differ in stack usage.

a) Space complexity and stack Usage.

Recursive Quick sort:

Recursive Quick sort uses the function call stack. Its average stack space is generally O(logn) when partitions are reasonably balanced. In the worst case, the recursion depth can become O(n)

Iterative Quick sort:

Iterative Quick sort removes function recursion and uses an explict stack or another mechanism to store pending sub arrays.

b) Ease of Implementation

Recursive Quick sort:

Recursive implementation is easier to understand and write because the algorithm naturally divides the problem into smaller sub problems.

Iterative Quick sort:

Iterative implementation is more complicated because the programmer must explicity manage the stack and store the ranges that still need to be sorted.

c) Risk of stack overflow:

Recursive Quick Sort:

there is a risk of stack overflow when partitions become highly unbalanced. In the worst case, recursion depth (can reach O(n).

Iterative Quick sort

It avoids the system's function-call recursion stack and therefore reduces the risk of traditional recursive stack overflow.

Comparision:

Feature	Recursive Quick sort	Iterative Quick sort
Implementation	Easier	More complex
stack usage	Function call stack	Explict stack
Average space	O(log n)	O(log n) typically.
worst-case space	O(n)	can be controlled.
stack overflow risk	Higher	lower.
Readability	High	Moderate